

Algoritmos Utilizados no Jogo Urban Chase (Unity / C#)

Este documento apresenta alguns algoritmos e estruturas de programação utilizadas no jogo “*Urban Chase*” desenvolvido na Unity. Os exemplos foram retirados diretamente dos scripts do projeto e explicados de forma simples.

Estrutura FOR

Arquivo de origem:	ArrastarItemScript.cs
--------------------	-----------------------

Trecho do código:

```
foreach (Collider c in cols)
    c.enabled = false;
```

Explicação:

O algoritmo de repetição FOR/FOREACH é utilizado para percorrer todos os objetos de uma lista. Nesse caso, o código percorre todos os colliders do objeto arrastado e desativa cada um deles. Isso evita colisões enquanto o item está sendo movido pelo jogador.

Estrutura WHILE / Atualização Contínua

Arquivo de origem:	ArrastarItemScript.cs
--------------------	-----------------------

Trecho do código:

```
void Update()
{
    if (!arrastando || objetoArrastando == null) return;
    AtualizarRotacaoDuranteArraste();
}
```

Explicação:

O método Update funciona como uma repetição contínua dentro da Unity. Enquanto o jogo estiver rodando, esse bloco é executado várias vezes por segundo. O IF verifica se o objeto está sendo arrastado antes de atualizar sua rotação.

Estrutura IF e ELSE

Arquivo de origem:	PoliciaScript.cs
--------------------	------------------

Trecho do código:

```
if (vertical > 0f)
    velocidadeAlvo = vertical * velocidadeMovimento;
else if (vertical < 0f)
    velocidadeAlvo = vertical * velocidadeRe;
```

Explicação:

A estrutura IF/ELSE é usada para tomar decisões. Nesse trecho, o jogo verifica se o jogador está acelerando para frente ou para trás. Dependendo da direção, uma velocidade diferente é aplicada ao carro policial.

Uso de ARRAY / LISTA

Arquivo de origem:	PolicialScript.cs
--------------------	-------------------

Trecho do código:

```
private Collider[] collidersPontoBloqueio;
```

Explicação:

Arrays servem para armazenar vários elementos do mesmo tipo. Nesse caso, o código guarda vários colliders relacionados às áreas de bloqueio. Isso permite verificar colisões em diferentes partes do cenário.

Detecção de Colisão

Arquivo de origem:	ColisaoSom.cs
--------------------	---------------

Trecho do código:

```
private void OnCollisionEnter(Collision collision)
{
    if (collision == null) return;
    if (!PodeTocar(collision.gameObject, collision.relativeVelocity.magnitude))
        return;
    TocarSom();
}
```

Explicação:

Esse algoritmo detecta quando ocorre uma colisão entre objetos. Quando a colisão acontece, o sistema verifica se pode tocar o som de impacto. Se as condições forem válidas, o áudio da batida é reproduzido.

Cálculo de Distância

Arquivo de origem:	FugitivoScript.cs
--------------------	-------------------

Trecho do código:

```
float distancia = Vector3.Distance(transform.position, policial.position);

if (distancia <= distanciaParaSerPego)
    CapturarFugitivo();
```

Explicação:

O algoritmo calcula a distância entre o fugitivo e o policial. Caso a distância seja menor que o limite definido, o fugitivo é capturado. Esse sistema é responsável pela mecânica principal do jogo.

Conclusão:

Os scripts do jogo utilizam diferentes algoritmos fundamentais da programação, como estruturas de repetição, condicionais, arrays, cálculos matemáticos e sistemas de colisão. Esses elementos são essenciais para criar a lógica do jogo e controlar o comportamento dos personagens e objetos.